
BINARY SEARCH TREE

A. KOMPETENSI DASAR

1. Memahami konsep dari Binary Search Tree.
2. Mengimplementasikan Binary Search Tree.
3. Memahami konsep menghapus node pada Binary Search Tree. Node yang dihapus adalah node root, node leaf, node yang mempunyai satu anak dan node yang mempunyai dua anak.
4. Mengimplementasikan menghapus node pada Binary Search Tree dengan menggunakan bahasa java

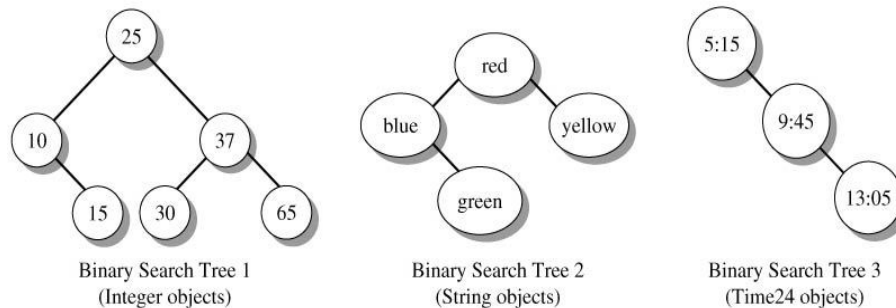
B. DASAR TEORI

Binary search tree (BST) adalah salah satu bentuk dari pohon. Di mana masing-masing pohon tersebut hanya memiliki dua buah upapohon, yakni upapohon kiri dan upapohon kanan. Ciri khas yang melekat pada binary search tree ini yang bisa juga dibidang sebagai keunggulan dari binary search tree adalah peletakan isi dari nodenya yang terurut berdasarkan besarnya dari isinya tersebut. Isinya bisa saja berupa integer, karakter, atau apapun sesuai dengan spesifikasi binary search tree yang ada. Operasi dasar dari binary search tree (BST) ini sendiri sangatlah sederhana, yakni hanya fungsi perbandingan dan fungsi rekursif. Di mana anak pohon sebelah kiri node adalah anak pohon yang lebih kecil dari node, sedangkan anak pohon sebelah kanan node memiliki isi yang lebih besar daripada isi node. Biasanya penyimpanan data di dalam binary search tree ini bisa juga berupa record di mana pengurutannya hanya tinggal melihat key dari record tersebut, misal nomor absen, NIM, tanggal, dll. Pada gambar 1 merupakan contoh Binary Search Tree, pada subtree kiri semua node mempunyai value lebih kecil dari root dengan value 25, sedangkan subtree kanan semua node mempunyai value lebih besar dari root dengan value 25. Pada Binary Search Tree, strategi penambahan Node baru adalah sebagai berikut:

- 1) Jika value dari node baru sama dengan value dari current node, maka mengembalikan nilai false.
- 2) Jika value dari node baru kurang dari value dari current node maka :
 - Jika anak kiri current node tidak null, maka ubah current node ke anak kiri tersebut, lakukan langkah 1.
 - Jika anak kiri current node adalah null, maka tambahkan node baru tersebut sebagai anak kiri dari current node

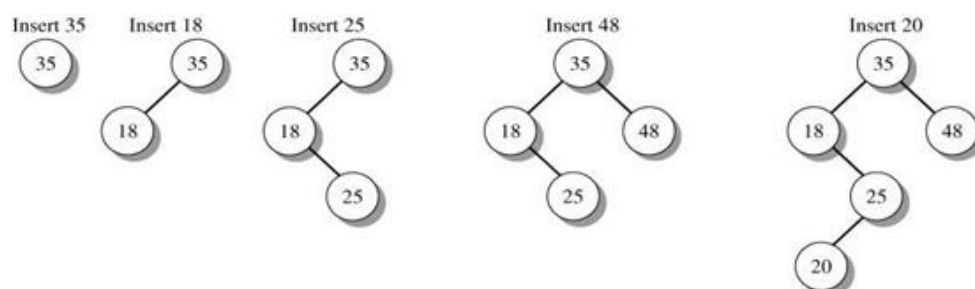
3) Jika value dari node baru lebih besar dari value dari current node maka :

- Jika anak kanan current node tidak null, maka ubah current node ke anak kanan tersebut, lakukan langkah 1.
- Jika anak kanan current node adalah null, maka tambahkan node baru tersebut sebagai anak kanan dari current node



Gambar 1. Contoh Binary Search Tree

Pada gambar 2 merupakan *step by step* penambahan node baru pada Binary Search Tree. Pertama ditambahkan node dengan value 35. Karena root masih null, maka node 35 menjadi root. Selanjutnya ditambahkan node 18. Pengecekan dimulai dari root, node 18 lebih kecil dibandingkan dengan node 35, dan node 35 tidak mempunyai anak kiri, sehingga node 18 menjadi anak kiri dari node 35. Selanjutnya ditambahkan node 25. Pengecekan dimulai dari root, node 25 lebih kecil dibandingkan dengan node 35, sehingga pengecekan selanjutnya ke anak kiri dari node 35 yaitu node 18. Node 25 lebih besar dibandingkan dengan node 18, karena node 18 tidak mempunyai anak kanan, sehingga node 25 menjadi anak kanan dari node 18. Selanjutnya ditambahkan node 48. Pengecekan dimulai dari root, node 48 lebih besar dibandingkan dengan node 35, karena node 35 tidak mempunyai anak kanan, sehingga node 48 menjadi anak kanan dari node 35. Selanjutnya ditambahkan node 20. Pengecekan dimulai dari root, node 20 lebih kecil dibandingkan dengan node 35, sehingga pengecekan selanjutnya ke anak kiri dari node 35 yaitu node 18. Node 20 lebih besar dibandingkan dengan node 18, sehingga pengecekan selanjutnya ke anak kanan dari node 18 yaitu node 25. Node 20 lebih kecil dibandingkan dengan node 25, karena node 25 belum mempunyai anak kiri, maka node 20 menjadi anak kiri dari node 25.



Gambar 2. Strategi Penambahan Node Baru pada Binary Search Tree

Search Method

Metode search adalah penentu paling dalam program atau bisa dibilang nyawa dari program database. Kenapa search sangat penting untuk efisien.? Bayangkan saja apabila ada miliaran data yang ada di dalam database pada sebuah komputer server dan harus diolah dan selalu harus diupdate setiap hari ataupun diakses pengguna untuk dicari isi data yang diinginkan. Lalu, yang kita gunakan adalah struktur data yang tidak efisien, di mana suatu saat computer server akan mengalami minimal hang atau yang paling parah adalah server down sehingga tidak bisa diakses dan akhirnya justru membuang waktu bahkan lebih parahnya bisa menyebabkan data dalam harddisk server menghilang. Tentunya kita sangat menghindari hal semacam itu terjadi bukan.

Metode Search sendiri ada banyak, seperti yang sudah sempat disinggung sedikit pada pendahuluan. Bahwa search macamnya ada *linear search*, *binary search*, *binary search tree*, dll. Di sini hanya akan dibahas 3 contoh tersebut. *Linear search* merupakan tipe search yang paling mendasar dalam dunia search. Pada dasarnya *linear search* ini sangat sederhana. Yakni memulai pencarian dari elemen pertama kemudian bergeser terus ke elemen berikutnya hingga menemukan isi elemen yang dicari. Apabila elemen yang dicari tidak ada, maka linear search akan terus melakukan traversal hingga elemen terakhir. Mari kita bayangkan apabila terdapat banyak sekali data yang harus ditraversal dan data yang dicari tidak ada di dalam database (worst case possibility). Efektivitas dari linear search ini adalah dengan $O(n)$ worst case tergantung dari jumlah elemen yang ada di dalam database tersebut. Metode search yang berikutnya yakni *binary search*, bedanya *binary search* dari linear search adalah dengan pembagian jumlah elemen menjadi dua bagian. Sehingga pencarian bisa lebih efektif dengan cara membagi pencarian ke dua arah. Worst case dari *binary search* ini adalah $O(\log n)$ sehingga cara ini bisa dibilang cukup efisien.

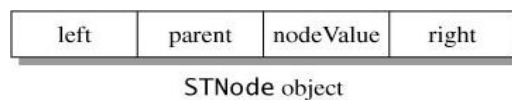
Metode search yang terakhir adalah *binary search tree* (sebenarnya bukan merupakan metode search). *Binary search tree* memiliki kompleksitas algoritma $O(\log n)$ yang tidak ada bedanya dengan metode binary search. Tetapi seharusnya metode search pada *binary search tree* bisa lebih singkat, karena jalur yang dipilih sudah tergolong spesifik, sehingga tidak perlu memeriksa semua elemen untuk mencari elemen tertentu.

Implementasi dari Binary Search Tree

Sebuah Node pada Binary Search Tree direpresentasikan dengan sebuah objek dari Class STNode seperti ditunjukkan pada gambar 3. Sebuah Node terdiri dari 4 bagian yaitu

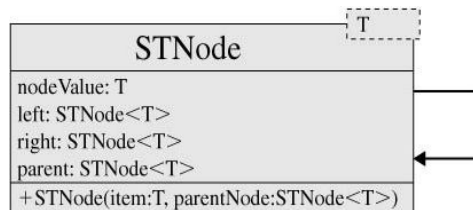
- Data value disebut nodeValue
- Variabel reference left sebagai anak kiri
- Variabel reference right sebagai anak kanan

- Variabel reference parent sebagai parent dari Node tersebut



Gambar 3. Objek dari Class STNode

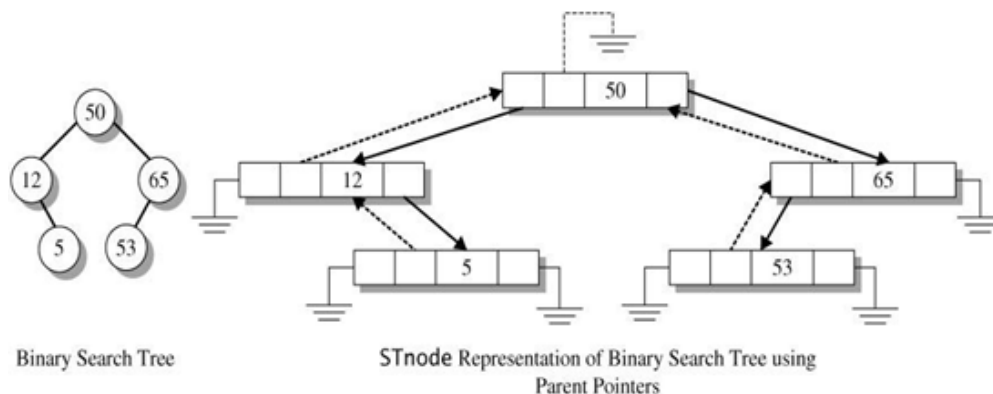
Pada gambar 4 adalah UML dari Class STNode<T>. Pada gambar 5 terdapat contoh bentuk tree dan bentuk Node-Node Tree yang direpresentasikan dengan objek STree.



Gambar 4. UML dari Class STNode<T>

```

1  class STNode<T> {
2      T nodeValue;
3      STNode<T> left, right, parent;
4
5      STNode(this.nodeValue, [this.parent]) {
6          left = null;
7          right = null;
8      }
9
10     STNode.withChildren(this.nodeValue, this.left, this.right) {
11         parent = null;
12     }
13 }
  
```

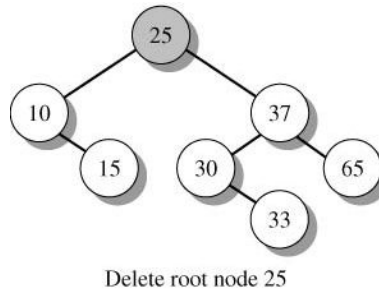


Gambar 5. Representasi STNode pada Binary Search Tree

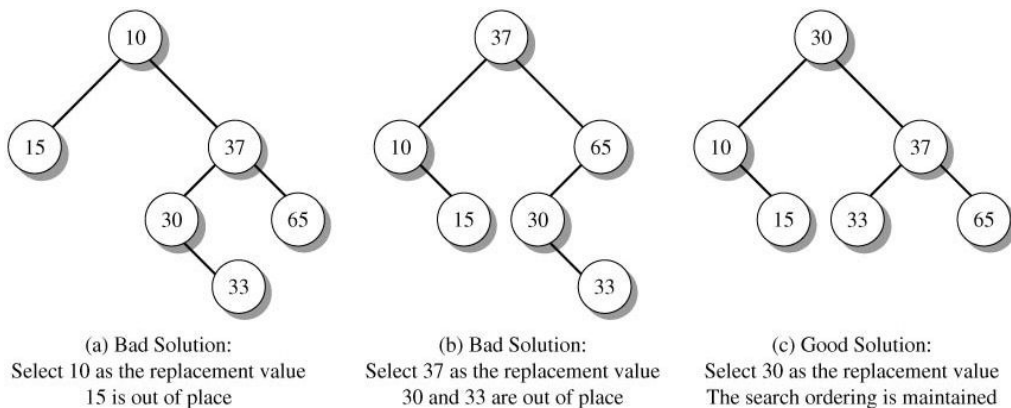
Menghapus Node pada Binary Search Tree

- 1) Aturan dalam menghapus node pada Binary Search Tree
- 2) Root pada BST tidak dapat dihapus, karena terdapat dua subtree pada node root. Tapi dapat diganti dengan node yang tepat yang terdapat pada BST.

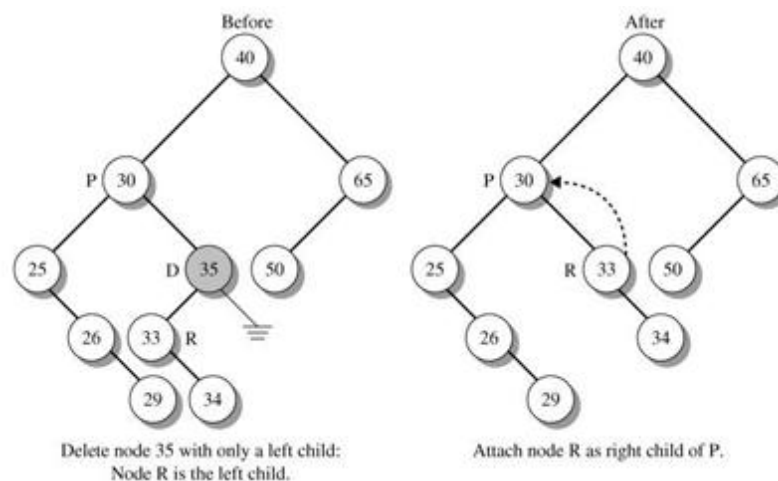
Sebagai contoh pada gambar 6, akan menghapus node root 25. Pada gambar 7(a), jika node 25 diganti dengan node 10, bukan merupakan solusi yang tepat, karena ada node yang tidak sesuai yaitu node 15. Pada gambar 7 (b), jika node 25 diganti dengan node 37, bukan merupakan solusi yang tepat, karena ada node yang tidak sesuai yaitu node 30 dan 33. Pada gambar 7(c), jika node 25 diganti dengan node 30, merupakan solusi yang tepat, karena semua node mengikuti aturan dari BST.



Gambar 6. Menghapus Node Root 25



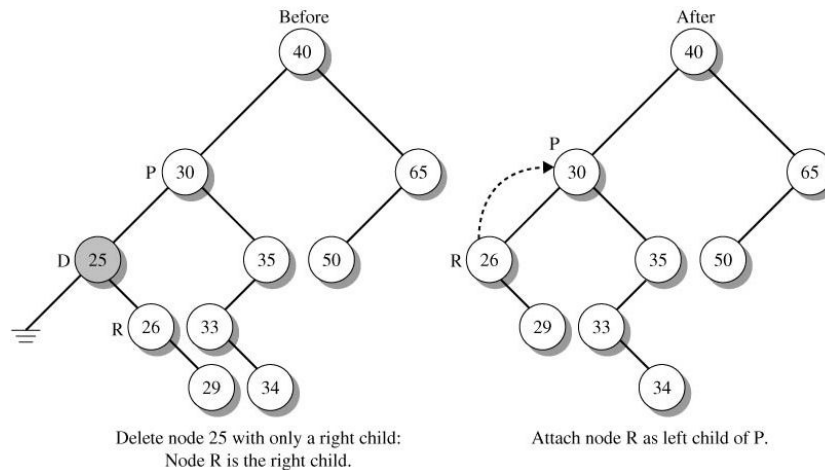
Gambar 7. Mencari pengganti dengan nilai yang tepat



Gambar 8. Contoh 1, menghapus node yang memiliki 1 anak

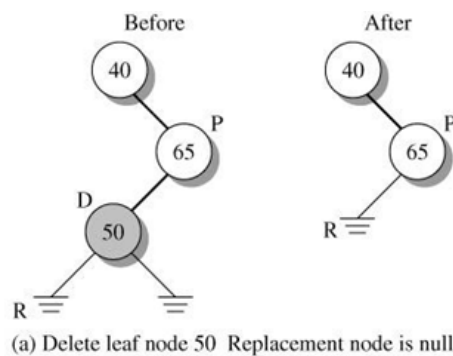
- 3) Menghapus sebuah node yang memiliki satu anak. Pada gambar 8 node 35 bukan node leaf, node 35 mempunyai subtree kosong. Node 35 diberi tanda dengan D, Parent dari node 35 yaitu

node 30 adalah P dan anak dari node 35 yaitu node 33 ditandai dengan R. Selanjutnya hapus node 35, dan kaitkan R(node 33) sebagai anak dari P(node 30). Pada gambar 4 node 25 bukan node leaf, node 25 mempunyai subtree kosong. Node 25 diberi tanda dengan D, Parent dari node 25 yaitu node 30 adalah P dan anak dari node 25 yaitu node 26 ditandai dengan R. Selanjutnya hapus node 25, dan kaitkan R(node 26) sebagai anak dari P(node 30).



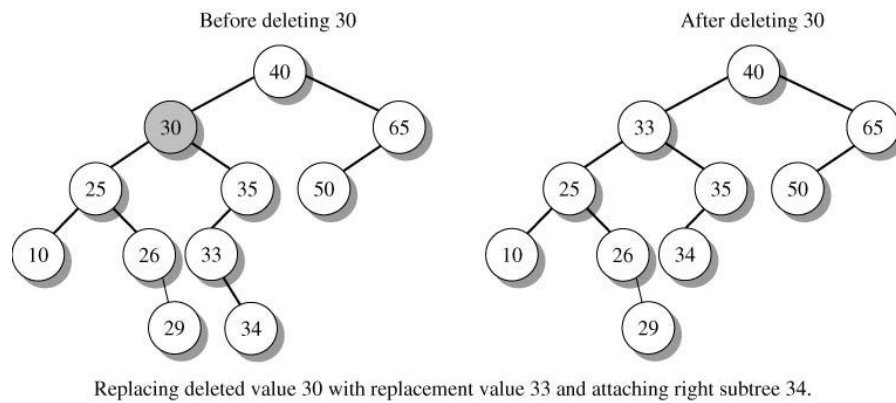
Gambar 9. Contoh 2, menghapus node yang memiliki 1 anak

- 4) Menghapus node leaf, menggunakan cara yang sama dengan menghapus node yang bukan leaf memiliki 1 anak. Pada gambar 10, node yang akan dihapus node 50 ditandai dengan D, parent dari node 50 yaitu node 65 ditandai dengan P, dan anak dari node 50 ditandai dengan R. R bernilai null, karena node yang akan dihapus adalah node leaf. Hapus node 50(D), kaitkan R(null) menjadi anak dari node P(65).



Gambar 10. Menghapus node leaf

- 5) Menghapus node yang memiliki dua anak. Pilih R (node yang berada di subtree dari node 30) sebagai node dengan nilai terkecil, tapi lebih besar dari nilai node yang dihapus. Pada gambar 11, node 30 sebagai node yang akan dihapus, cari node R yang terkecil, tapi lebih besar dari nilai node yang dihapus (node 30) yaitu node 33. Hapus node 30 dan kaitkan R ke parent dari node yang dihapus.



Gambar 11. Menghapus node yang memiliki dua anak

C. TUGAS PENDAHULUAN

1. Buatlah resume 1 halaman mengenai Binary Search Tree
2. Buatlah Binary Search Tree dari node-node 30, 15, 17, 36, 67, 30, 69

D. PERCOBAAN

Pada percobaan ini, buatlah *class* Binary Search Tree seperti pada tabel berikut.

Tabel 1. Variabel dan Method pada *Class* Binary Search Tree

<i>Class Binary Search Tree <T></i>	
STNode<T> root;	Variabel reference ke root
int treeSize;	Variable untuk mengetahui jumlah node pada binary search tree
bool add (T item) {}	Method add() untuk menambahkan node baru ke binary search tree. Mengembalikan nilai true jika node baru berhasil ditambahkan, mengembalikan nilai false jika node baru sudah terdapat pada binary search tree.
STNode<T> findNode(T item)	Method findNode() untuk mencari sebuah node dengan item tertentu dengan mengembalikan alamat dari node tersebut.
bool find(T item)	Method find() untuk mencari sebuah node dengan item tertentu dengan mengembalikan nilai true jika ada item tersebut, nilai false jika tidak ada.
T first() { }	Method first() untuk mendapatkan nilai terkecil dari node-node yang terdapat pada binary search tree.
T last() { }	Method last() untuk mendapatkan nilai terbesar dari node-node yang terdapat pada binary search tree
STNode<T> getRoot() { }	Method getRoot() untuk mendapatkan alamat reference dari root
int getTreeSize() { }	Method getTreeSize() untuk mendapatkan jumlah node pada Binary Search Tree

void removeNode (STNode<T> dNode)	Method yang digunakan untuk menghapus node pada Binary search tree
---	---

Percobaan 1 : Membuat class `BinarySearchTree<T>`, konstruktor dan method `add()`.
 Method `add()` untuk menambahkan node baru ke binary search tree.
 Mengembalikan nilai `true` jika node baru berhasil ditambahkan, mengembalikan nilai
`false` jika node baru sudah terdapat pada binary search tree.

```
class BinarySearchTree<T> {
    STNode<T>? root;
    int treeSize = 0;

    BinarySearchTree() {
        root = null;
    }

    bool add(T item) {
        STNode<T> t = root, parent;
        int orderValue = 0;

        while (t != null) {
            parent = t;
            orderValue = item.compareTo(t.nodeValue);

            if (orderValue == 0) {
                return false;
            } else if (orderValue < 0) {
                t = t.left;
            } else {
                t = t.right;
            }
        }

        STNode<T> newNode = STNode(item, parent);
        if (parent == null) {
            root = newNode;
        } else if (orderValue < 0) {
            parent.left = newNode;
        } else {
            parent.right = newNode;
        }
        treeSize++;
        return true;
    }
}
```

Percobaan 2 : Membuat method `findNode()`, untuk mencari sebuah node dengan item tertentu
 dengan mengembalikan alamat dari node tersebut

```
class BinarySearchTree<T> {
    ...
    STNode<T> findNode(T item) {
        STNode<T> t = root, parent = null;
        int orderValue = 0;
```



```

        while (t != null) {
            orderValue = item.compareTo(t.nodeValue);

            if (orderValue == 0) {
                return t;
            } else if (orderValue < 0) {
                t = t.left;
            } else {
                t = t.right;
            }
        }
        return null;
    }
}

```

Percobaan 3 : Membuat method `find()`, untuk mencari sebuah node dengan item tertentu dengan mengembalikan nilai *True* jika ada item tersebut, nilai *False* jika tidak ada.

```

class BinarySearchTree<T> {
    ...
    bool find(T item) {
        STNode<T> t = findNode(item);

        if (t != null) {
            T value = t.nodeValue;
            return true;
        }
        return false;
    }
}

```

Selanjutnya, ujlilah program yang sudah dibuat dengan *Class Main* seperti di bawah ini:

```

void main() {
    BinarySearchTree<dynamic> bst = BinarySearchTree();
    bst.add(35);
    bst.add(18);
    bst.add(25);
    bst.add(48);
    bst.add(20);

    print('Tree Size: ${bst.treeSize}');

    var node = bst.findNode(5);
    if (node != null) {
        print('Node found with value: ${node.nodeValue}');
    } else {
        print('Node not found. ');
    }
    bool node = bst.find(5);
    if (node) {
        print('Node found');
    } else {
        print('Node not found. ');
    }
}

```

Berdasarkan percobaan 1-3 di atas, anda telah mencoba untuk membuat sebuah operasi *Binary Search Tree* dengan menambahkan class `BinarySearchTree()` pada syntax sebelumnya. Pada syntax tersebut, terdapat method `add()` untuk menambahkan data pada BST, method `findNode()` dan `find()` untuk mencari nilai pada tree. Selanjutnya syntax di atas, tambahkan untuk mendapatkan nilai pada `rootNode` dengan menambahkan method `STNode<T> getRoot()` dan Method `InOrderDisplay(STNode<T>? node)`, `PreOrderDisplay(STNode<T>? node)`, dan `PostOrderDisplay(STNode<T>? node)` untuk mencetak atau mengakses nilai pada BST (Traversal BST)! (Syntax Traversal sama dengan praktikum Binary Tree sebelumnya! Jalankan method tersebut!

Percobaan 4 : Method untuk menghapus sebuah Node

```

1  class BinarySearchTree<T> {
2      STNode<T> root;
3      int treeSize;
4      ...
5
6      void RemoveNode(STNode<T>? dNode) {
7          if (dNode == null) {
8              return;
9          }
10         STNode<T>? pNode, rNode;
11         pNode = dNode.parent;
12         // Node yang dihapus mempunyai satu anak
13         if (dNode.left == null || dNode.right == null) {
14             if (dNode.right == null) {
15                 rNode = dNode.left;
16             } else {
17                 rNode = dNode.left;
18             }
19
20             if (rNode != null) {
21                 print("Ngeset Parent");
22                 rNode.parent = pNode;
23             }
24             // Menghapus node root
25             if (pNode == null) {
26                 root = rNode;
27             } else if ((dNode.nodeValue as Comparable<T>).compareTo
28 (pNode.nodeValue) < 0) {
29                 pNode.left = rNode;
30             } else {
31                 pNode.right = rNode;
32             }
33         } // Node yang dihapus mempunyai dua anak
34         else {
35             STNode<T>? pOfRNode = dNode;
36             rNode = dNode.right;
37             pOfRNode = dNode;
38
39             while (rNode!.left != null) {
40                 pOfRNode = rNode;
41                 rNode = rNode.left;
42             }
43             dNode.nodeValue = rNode.nodeValue;

```

```

44     if (pOfRNode == dNode) {
45         dNode.right = rNode.right;
46     } else {
47         pOfRNode!.left = rNode.right;
48     }
49
50     if (rNode.right != null) {
51         rNode.right!.parent = pOfRNode;
52     }
53 }
54 }
55 }

```

Selanjutnya ujliah program di atas dengan *Class Main* sebagai berikut

```

1  void main() {
2      BinarySearchTree<dynamic> bst = BinarySearchTree();
3      bst.add(35);
4      bst.add(18);
5      bst.add(25);
6      bst.add(48);
7      bst.add(20);
8
9      STNode<dynamic>? rootNode = bst.getRoot();
10     if (rootNode != null) {
11         print('Root Value: ${rootNode.nodeValue}');
12     } else {
13         print('The tree is empty.');
```

E. LATIHAN

1. Pada percobaan untuk fungsi Penghapusan node di atas, langkah penghapusan dilakukan dengan mengganti langsung node yang dihapus dengan node pengganti. Modifikasilah method tersebut dimana node yang akan diganti dihapus terlebih dahulu sebelum digantikan dengan node lain (seperti pada percobaan binary tree sebelumnya). Dan jelaskan langkah-langkah dari method `removeNode` tersebut!
2. Buatlah method `first()` untuk mendapatkan nilai terkecil dari node-node yang terdapat pada Binary search tree.
3. Buatlah method `last()` untuk mendapatkan nilai terbesar dari node-node yang terdapat pada Binary search tree.

4. Buatlah Binary search tree dengan menambahkan node dengan value {10, 5,20,2,6,19,25,21}.

Kerjakan :

- Gambarkan hasil akhir binary search tree tersebut !
 - Bacalah Binary Search Tree dengan Traversal PostOrder
 - Cari node dengan nilai 19.
 - Cari node dengan nilai 55.
 - Tambahkan pada Binary Tree Node baru 56.
 - Dapatkan node dengan nilai terkecil
 - Dapatkan node dengan nilai terbesar
5. Kerjakan studi kasus yang terdapat pada gambar 6 s/d 11!
6. Dari studi kasus latihan 4, jelaskan langkah-langkahnya berdasarkan program!

F. LAPORAN RESMI

Kerjakan hasil percobaan(D) dan latihan(E) di atas dan tambahkan analisa.

Format laporan praktikum Judul, Tujuan Praktikum, Tugas Pendahuluan, Percobaan, Latihan, Kesimpulan dan Analisis, Referensi. Dikumpulkan di lms Poliwangi dengan nama file Kelas_TigaAngkaNIM dari belakang_Nama_Nama Praktikum.pdf